

PROJECT 3
LOGISTIC REGRESSION, K-NEAREST NEIGHBOR,
NAÏVE BAYES, DECISION TREE

disusun untuk memenuhi tugas mata
kuliah *Pembelajaran Mesin*

oleh:

Arif Maulana	(2208107010067)
M. Nouval Rifqi	(2208107010075)
Azzariyat Azra	(2208107010079)
Tiara Agustin	(2208107010004)



JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
2025

1) Pemahaman Dataset

a) Sumber Data

Dataset yang digunakan dalam laporan ini adalah *Heart Failure Prediction Dataset* yang diunduh dari [Kaggle](#). Dataset ini bertujuan untuk memprediksi kemungkinan gagal jantung berdasarkan berbagai fitur klinis pasien.

b) Statistik Deskriptif dan Visualisasi Awal Data

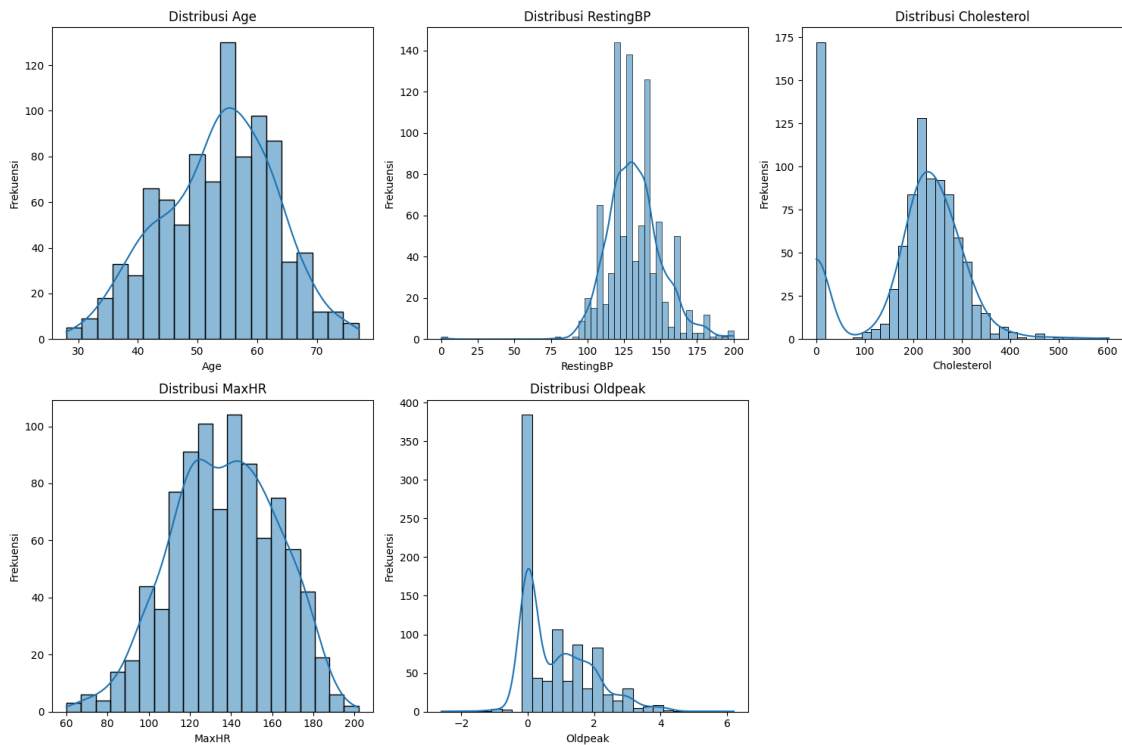
- i) Berikut adalah statistik deskriptif dari beberapa fitur numerik utama dalam dataset:

Fitur	Mean	Std Dev	Min	25%	Median	75%	Max
Age	53.51	9.43	28	47	54	60	77
RestingBP	132.95	18.51	0	120	130	140	200
Cholesterol	198.80	109.34	0	173	223	267	603
MaxHR	136.89	25.46	60	120	138	156	202
Oldpeak	0.88	1.06	-2	0	0.6	1.5	6.2

Interpretasi awal:

- **Age** memiliki rata-rata sekitar 54 tahun, dengan rentang usia dari 28 hingga 77 tahun.
- Terdapat nilai **nol** pada fitur *RestingBP* dan *Cholesterol*, yang kemungkinan besar merupakan nilai tidak valid (anomalies) dan akan diperhatikan pada tahap pra-pemrosesan.
- Fitur *Oldpeak*, yang menunjukkan depresi ST, memiliki nilai negatif yang perlu dianalisis apakah valid secara medis.
- *MaxHR* berkisar antara 60 hingga 202, dengan nilai tengah 138.

ii) Visualisasi Awal Data

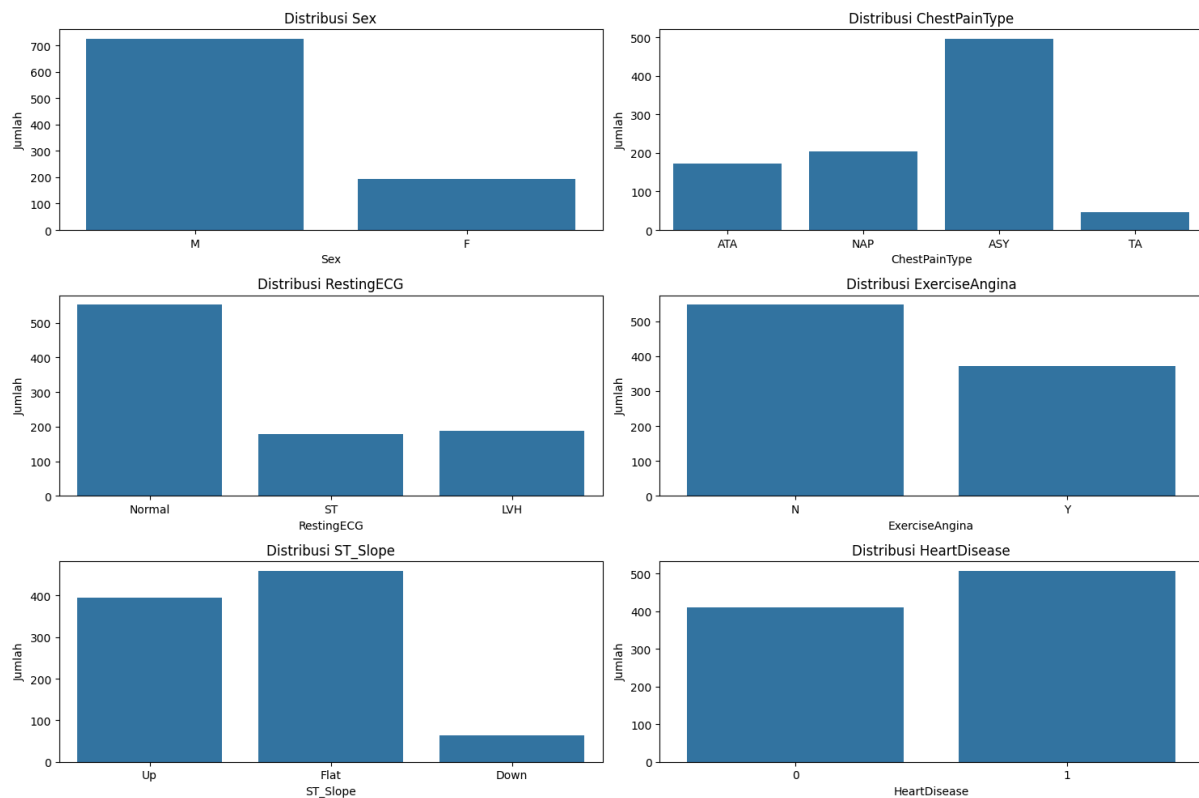


Gambar 1.1 Histogram untuk fitur numerik

Visualisasi awal digunakan untuk memahami distribusi masing-masing fitur numerik secara visual. Gambar berikut menunjukkan histogram dari masing-masing fitur:

Interpretasi:

- **Age** dan **MaxHR** menunjukkan distribusi yang cenderung normal (mendekati simetris).
- **RestingBP** dan **Cholesterol** menunjukkan distribusi yang skew ke kanan, dengan kemungkinan nilai-nilai ekstrem.
- **Oldpeak** memiliki distribusi yang sangat skew ke kanan, dengan mayoritas nilai berada di kisaran rendah (0–2), namun terdapat beberapa nilai tinggi hingga lebih dari 6.



Gambar 1.2 CountPlot

Visualisasi distribusi fitur kategorikal bertujuan untuk memahami karakteristik populasi dalam dataset dan potensi imbalance pada beberapa fitur.

1. Sex:

- Mayoritas data berasal dari pasien laki-laki (M) sekitar 700+ orang, sedangkan perempuan (F) jauh lebih sedikit, sekitar 200-an.

2. ChestPainType:

- Tipe nyeri dada ASY (Asymptomatic) mendominasi secara signifikan.
- Tipe lainnya seperti ATA, NAP, dan TA memiliki jumlah jauh lebih rendah, terutama TA.

3. RestingECG:

- Mayoritas menunjukkan hasil EKG normal.
- Dua kategori lainnya (ST, LVH) memiliki jumlah yang seimbang dan lebih sedikit dibandingkan Normal.

4. ExerciseAngina:

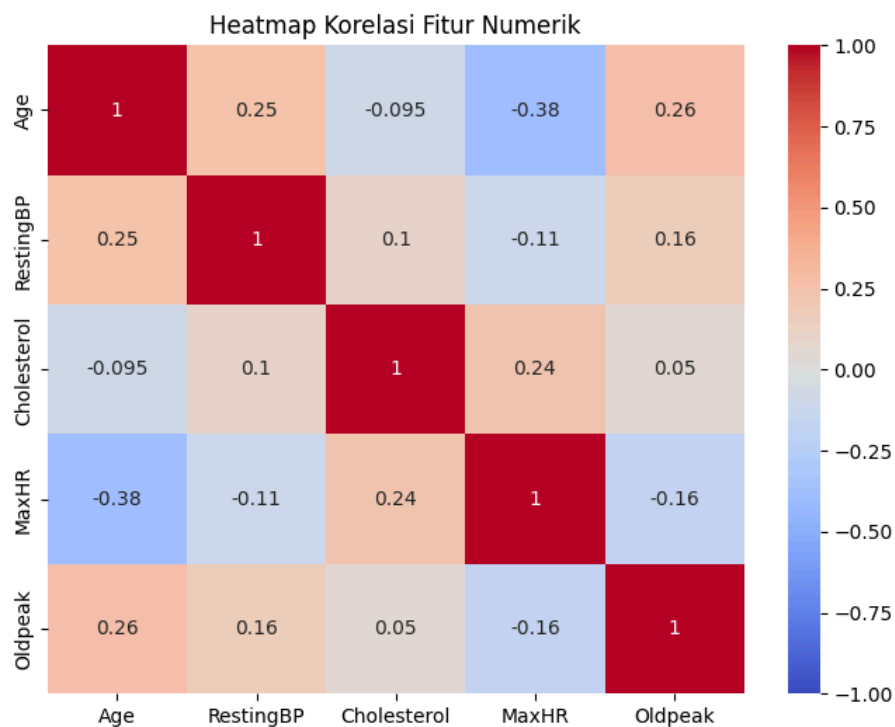
- Sebagian besar pasien tidak mengalami angina saat olahraga (N).
- Sekitar 40% mengalami angina saat berolahraga (Y).

5. ST_Slope:

- Kategori Flat paling umum, disusul Up, dan Down menjadi yang paling jarang.

6. HeartDisease (Target):

- Distribusi target relatif seimbang, dengan sedikit lebih banyak pasien yang positif penyakit jantung (1) dibandingkan yang tidak (0).
- Hal ini baik untuk klasifikasi karena menghindari ketimpangan kelas (class imbalance) yang ekstrem.



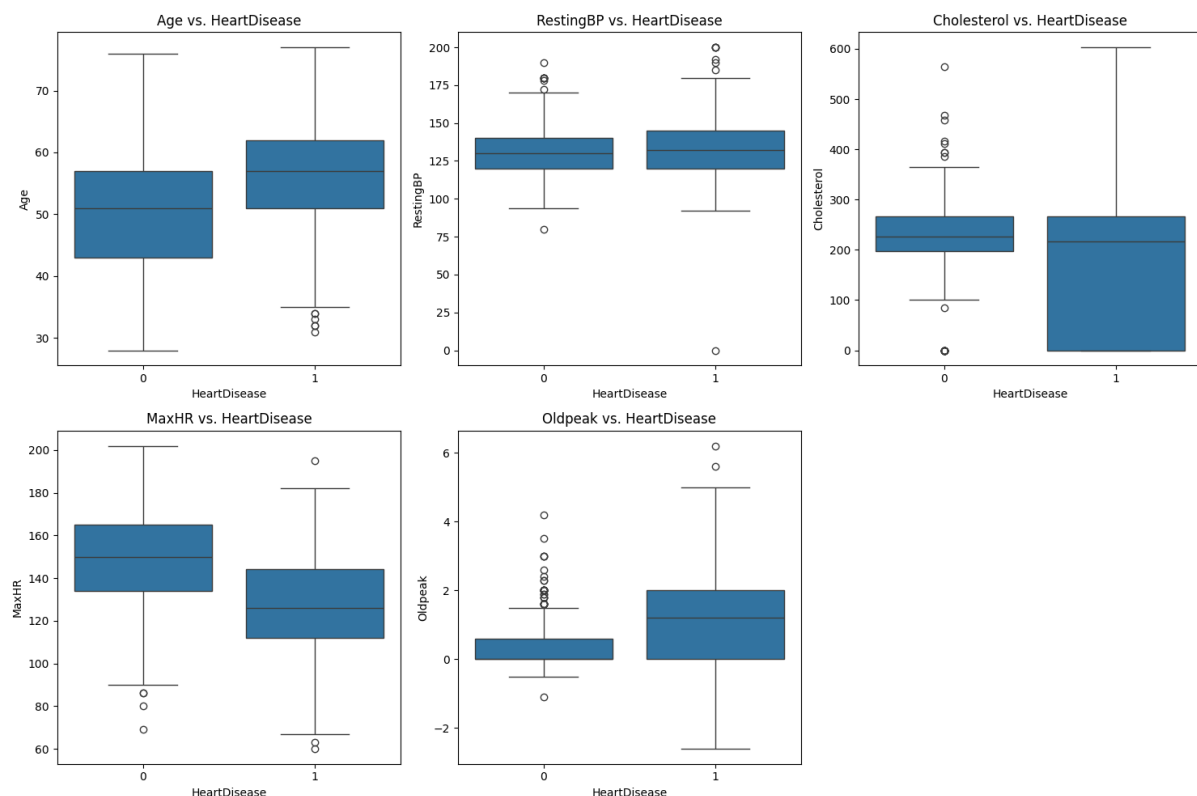
Gambar 1.3 Heatmap

Untuk memahami hubungan antar fitur numerik dalam dataset, digunakan visualisasi **heatmap korelasi**. Korelasi diukur menggunakan koefisien Pearson, dengan rentang nilai dari -1 (korelasi negatif sempurna) hingga 1 (korelasi positif sempurna).

Interpretasi Korelasi:

- Korelasi tertinggi bernilai **positif** terjadi antara Age dan Oldpeak (0.26), serta RestingBP dan Age (0.25). Artinya, seiring bertambahnya usia, kemungkinan nilai Oldpeak dan RestingBP juga meningkat secara moderat.
- Hubungan negatif cukup jelas terlihat antara Age dan MaxHR (-0.38), yang secara medis logis karena detak jantung maksimum cenderung menurun seiring bertambahnya usia.
- Tidak terdapat korelasi kuat (≥ 0.7 atau ≤ -0.7) antara fitur-fitur numerik, menunjukkan tidak adanya multikolinearitas yang tinggi antar fitur.

Visualisasi ini membantu dalam menentukan fitur mana yang berpotensi relevan dalam proses pemodelan selanjutnya. Fitur dengan korelasi rendah terhadap fitur lainnya tetap dapat dipertimbangkan, terutama jika menunjukkan hubungan dengan variabel target.



Gambar 1.4 Boxplot

1. Age vs. HeartDisease

- Pasien dengan penyakit jantung cenderung lebih tua dibandingkan yang tidak menderita.
- Median usia pasien dengan HeartDisease 1 sedikit lebih tinggi daripada yang 0.

2. RestingBP vs. HeartDisease

- Tekanan darah istirahat (RestingBP) tidak menunjukkan perbedaan signifikan antara keduanya.
- Namun, terdapat outlier ekstrem di HeartDisease 1 (nilai 0), yang bisa jadi kesalahan pencatatan.

3. Cholesterol vs. HeartDisease

- Distribusi kolesterol sangat tersebar, khususnya pada kelas HeartDisease 1.
- Ada banyak nilai outlier pada keduanya, dan beberapa nilai ekstrem nol (yang mungkin perlu ditinjau atau diimputasi ulang).

4. MaxHR vs. HeartDisease

- Pasien yang **tidak** memiliki penyakit jantung (0) cenderung memiliki detak jantung maksimum (MaxHR) lebih tinggi.
- Ini sejalan dengan fakta bahwa pasien sehat bisa berolahraga lebih keras hingga detak jantungnya tinggi.

5. Oldpeak vs. HeartDisease

- Pasien dengan penyakit jantung (1) cenderung memiliki nilai Oldpeak lebih tinggi.
- Oldpeak mengukur depresi ST setelah olahraga; semakin tinggi, semakin mungkin ada iskemia miokard.

Insight Umum:

- Fitur numerik yang menunjukkan potensi pengaruh signifikan terhadap penyakit jantung:
 - **MaxHR** (lebih rendah pada pasien sakit)
 - **Oldpeak** (lebih tinggi pada pasien sakit)

- **Age** (pasien sakit cenderung lebih tua)

2) Eksplorasi Data dan Pra-pemrosesan

c) Penanganan Nilai 0 RestingBP dan Cholestrol

```
Jumlah nilai 0 sebelum penggantian:  
RestingBP: 1  
Cholesterol: 172  
  
Jumlah nilai 0 setelah penggantian:  
RestingBP: 0  
Cholesterol: 0
```

Gambar 2.1 Penanganan nilai 0 dan Resting BP dan Cholestrol

Dalam proses eksplorasi data awal, ditemukan adanya nilai 0 (nol) pada dua fitur numerik, yaitu RestingBP (tekanan darah saat istirahat) dan Cholesterol (kadar kolesterol).

Secara medis, nilai 0 pada kedua fitur tersebut tidak valid karena:

- RestingBP (Resting Blood Pressure) tidak mungkin bernilai 0 kecuali seseorang mengalami kondisi kritis (yang tidak relevan dalam konteks data ini).
- Cholesterol juga tidak mungkin bernilai 0 karena semua manusia memiliki kadar kolesterol dalam darah.

Jumlah Nilai 0 yang Ditemukan:

- d) RestingBP: 1 nilai nol
- e) Cholesterol: 172 nilai nol

Hasil Setelah Penggantian

- Jumlah nilai nol pada RestingBP setelah penggantian: **0**
- Jumlah nilai nol pada Cholesterol setelah penggantian: **0**

f) One-Hot Encoding

Dalam dataset ini, terdapat beberapa fitur dengan tipe data kategorikal yang perlu diubah menjadi bentuk numerik agar dapat digunakan dalam model pembelajaran mesin. Model seperti regresi logistik, K-Nearest Neighbors (KNN), dan pohon keputusan membutuhkan input numerik.

Fitur Kategorikal yang Ditangani:

Berikut adalah daftar fitur kategorikal yang dikonversi:

- Sex
- ChestPainType
- RestingECG
- ExerciseAngina
- ST_Slope

Metode Encoding

Metode yang digunakan untuk mengubah fitur-fitur tersebut adalah One-Hot Encoding, yaitu dengan membuat kolom baru untuk setiap kategori, dan memberikan nilai 1 jika baris tersebut termasuk dalam kategori tersebut, dan 0 jika tidak.

Penggunaan `drop_first=True` dilakukan untuk menghindari multikolinearitas, yaitu masalah ketika fitur yang dihasilkan dari encoding menjadi saling berkorelasi secara linear.

Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease	Sex_M	ChestPainType_ATA
40	140	289	0	172	0.0	0	True	True
49	160	180	0	156	1.0	1	False	False
37	130	283	0	98	0.0	0	True	True
48	138	214	0	108	1.5	1	False	False
54	150	195	0	122	0.0	0	True	False

Gambar 2.2 Output dari metode One-hot Encoding

Hasil Transformasi

Contoh hasil transformasi (5 baris pertama) menunjukkan bahwa fitur-fitur seperti:

- Sex_M menggantikan Sex

- ChestPainType_ATA, ChestPainType_NAP, ChestPainType_TA menggantikan ChestPainType
- Dan seterusnya untuk fitur lainnya.

Nilai dalam kolom tersebut berupa **Boolean (True/False)** atau bisa juga berupa integer biner (1/0) tergantung konfigurasi saat ekspor/tampilan data.

g) Normalisasi/Standarisasi

Dalam proses *data preprocessing*, penting untuk menyesuaikan skala data numerik agar model pembelajaran mesin dapat berfungsi optimal. Hal ini sangat dibutuhkan terutama untuk algoritma yang sensitif terhadap skala fitur, seperti *K-Nearest Neighbors*, *SVM*, dan *regresi logistik*.

Fitur Numerik yang Dinormalisasi

Fitur yang termasuk numerik dan telah dinormalisasi:

- Age
- RestingBP (tekanan darah saat istirahat)
- Cholesterol
- MaxHR (detak jantung maksimal)
- Oldpeak (depresi ST yang diukur dalam latihan)

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease	Sex_M	ChestPainType_ATA	ChestPainType_NAP
0	-1.433140	0.415002	0.897407	0	1.382928	-0.832432	0	True	True	False
1	-0.478484	1.527329	-1.122849	0	0.754157	0.105664	1	False	False	True
2	-1.751359	-0.141161	0.786200	0	-1.525138	-0.832432	0	True	True	False
3	-0.584556	0.303769	-0.492677	0	-1.132156	0.574711	1	False	False	False
4	0.051881	0.971166	-0.844832	0	-0.581981	-0.832432	0	True	False	True

Gambar 2.3 Output setelah di normalisasi

Hasil Transformasi

Seperti terlihat pada gambar, nilai dari kelima fitur tersebut sekarang berada dalam rentang skala normalisasi (sekitar -3 hingga +3). Ini membantu model untuk lebih cepat berkonvergensi saat pelatihan dan meningkatkan akurasi prediksi.

3) Implementasi Model

a) Model Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, log_loss
import seaborn as sns
import matplotlib.pyplot as plt

# Inisialisasi dan melatih model
logreg = LogisticRegression(random_state=42)
logreg.fit(X_train, y_train)

# Membuat prediksi
y_pred_logreg = logreg.predict(X_test)
y_pred_prob_logreg = logreg.predict_proba(X_test)

# Menghitung Confusion Matrix
cm_logreg = confusion_matrix(y_test, y_pred_logreg)

# Visualisasi Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm_logreg, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Menampilkan Classification Report
print("Classification Report - Logistic Regression:")
print(classification_report(y_test, y_pred_logreg))

# Menghitung loss Value (Log Loss)
loss_logreg = log_loss(y_test, y_pred_prob_logreg)
print(f"Log Loss - Logistic Regression: {loss_logreg:.4f}")
```

Gambar 3.1 Implementasi model Logistic Regression

Implementasi model Logistic Regression dimulai dengan mengimpor class LogisticRegression dari sklearn.linear_model. Logistic Regression dipilih karena merupakan algoritma yang sederhana namun efektif untuk masalah klasifikasi biner seperti mendeteksi apakah seseorang memiliki penyakit jantung atau tidak. Model ini bekerja dengan menghitung probabilitas dan memetakan input fitur ke dalam dua kelas menggunakan fungsi sigmoid.

Selanjutnya, model diinisialisasi dengan parameter max_iter=1000. Parameter ini menentukan jumlah maksimum iterasi saat proses pelatihan berlangsung. Angka ini dinaikkan dari default-nya untuk menghindari masalah non-konvergensi yang kadang muncul ketika data cukup kompleks atau memiliki banyak fitur hasil dari one-hot encoding.

Setelah model siap, dilakukan pelatihan menggunakan model.fit(X_train, y_train), yang berarti model belajar dari data latih. Kemudian, prediksi terhadap data uji dilakukan menggunakan model.predict(X_test). Hasil prediksi inilah yang kemudian digunakan untuk mengevaluasi performa model, misalnya melalui confusion matrix seperti yang ditampilkan pada grafik sebelumnya.

Dengan pendekatan ini, Logistic Regression memberikan baseline yang baik untuk membandingkan performa dengan model lain.

b) Model K-NN

```
from sklearn.neighbors import KNeighborsClassifier

# Inisialisasi dan melatih model
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Membuat prediksi
y_pred_knn = knn.predict(X_test)

# Menghitung Confusion Matrix
cm_knn = confusion_matrix(y_test, y_pred_knn)

# Visualisasi Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - K-NN')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Menampilkan Classification Report
print("Classification Report - K-NN:")
print(classification_report(y_test, y_pred_knn))
```

Gambar 3.2 Implementasi model K-NN

Model K-Nearest Neighbors (K-NN) digunakan sebagai alternatif pendekatan non-parametrik yang mengklasifikasikan data berdasarkan kedekatan dengan data lain di sekitarnya. K-NN tidak melakukan proses pelatihan secara eksplisit seperti model lainnya, melainkan menyimpan seluruh data latih dan menggunakan prinsip jarak (biasanya Euclidean) untuk menentukan kelas dari data uji.

Pada implementasi ini, model diinisialisasi dengan `n_neighbors=5`, yang berarti untuk menentukan kelas suatu data uji, model melihat 5 tetangga terdekatnya di data latih. Pemilihan angka 5 adalah nilai umum yang sering digunakan untuk memulai, dan bisa disesuaikan dengan proses tuning parameter.

Setelah model di-fit dan digunakan untuk memprediksi data uji, hasil prediksi dianalisis menggunakan confusion matrix dan classification report. Confusion matrix memberikan gambaran visual mengenai jumlah prediksi benar dan salah, sedangkan classification report menunjukkan metrik evaluasi seperti precision, recall, f1-score, dan akurasi dari model K-NN ini. Pendekatan ini cocok untuk dataset dengan struktur yang jelas atau ketika kita ingin model yang intuitif dan mudah dipahami.

c) Model Naive Bayes

```
from sklearn.naive_bayes import GaussianNB

# Inisialisasi dan melatih model
nb = GaussianNB()
nb.fit(X_train, y_train)

# Membuat prediksi
y_pred_nb = nb.predict(X_test)

# Menghitung Confusion Matrix
cm_nb = confusion_matrix(y_test, y_pred_nb)

# Visualisasi Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Naïve Bayes')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Menampilkan Classification Report
print("Classification Report - Naïve Bayes:")
print(classification_report(y_test, y_pred_nb))
```

Gambar 3.3 Implementais model Naive Bayes

Model Naïve Bayes, khususnya varian GaussianNB, digunakan karena sangat efisien dan cocok untuk data numerik yang diasumsikan berdistribusi normal. Model ini bekerja dengan prinsip probabilistik, yaitu menghitung kemungkinan suatu data termasuk ke dalam kelas tertentu berdasarkan fitur-fiturnya, dengan asumsi bahwa fitur-fitur tersebut bersifat independen (naïve assumption).

Pada implementasi ini, GaussianNB() digunakan karena sebagian besar fitur telah dinormalisasi dan dianggap mengikuti distribusi Gaussian. Model dilatih menggunakan data latih, kemudian dipakai untuk memprediksi label pada data uji. Evaluasi dilakukan dengan confusion matrix untuk melihat jumlah prediksi benar/salah, serta classification report untuk melihat metrik seperti precision, recall, dan f1-score.

Naïve Bayes sering digunakan karena cepat, sederhana, dan cukup efektif untuk masalah klasifikasi dasar, terutama saat jumlah data besar dan tidak memerlukan banyak parameter tuning.

d) Model Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

# Inisialisasi dan melatih model
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

# Membuat prediksi
y_pred_dt = dt.predict(X_test)

# Menghitung Confusion Matrix
cm_dt = confusion_matrix(y_test, y_pred_dt)

# Visualisasi Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Decision Tree')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Menampilkan Classification Report
print("Classification Report - Decision Tree:")
print(classification_report(y_test, y_pred_dt))
```

Gambar 3.4 Implementasi model Decision Tree

Model Decision Tree digunakan karena mampu menangani fitur kategorikal maupun numerik secara langsung, serta memberikan interpretasi yang jelas dalam bentuk struktur pohon keputusan. Algoritma ini bekerja dengan membagi data ke dalam cabang-cabang

berdasarkan fitur yang memberikan informasi paling besar (biasanya menggunakan kriteria seperti Gini Impurity atau Entropy).

Dalam implementasi ini, model diinisialisasi menggunakan `DecisionTreeClassifier` dengan parameter `random_state=42` untuk memastikan hasil yang konsisten. Model kemudian dilatih dengan data latih dan digunakan untuk memprediksi kelas pada data uji.

Evaluasi dilakukan dengan confusion matrix untuk melihat distribusi prediksi benar dan salah, serta classification report yang memberikan insight metrik performa seperti precision, recall, dan f1-score. Decision Tree cocok untuk memahami pola dalam data secara visual, dan juga berguna sebagai dasar bagi model-model ensemble seperti Random Forest.

e) Teknik Validasi

Untuk mengukur performa model dan mencegah overfitting, digunakan teknik validasi Train-Test Split. Teknik ini membagi dataset menjadi 2 bagian:

- Data latih (80%) digunakan untuk melatih model.
- Data uji (20%) digunakan untuk mengevaluasi kinerja model.

Pembagian ini dilakukan secara acak dan diterapkan secara konsisten pada setiap algoritma yang diimplementasikan (Logistic Regression, K-NN, Naïve Bayes, dan Decision Tree). Cara ini digunakan agar hasil evaluasi antar model dapat dibandingkan secara adil.

Penggunaan train-test split ini bertujuan untuk mencegah overfitting dan memastikan bahwa model tidak hanya menghafal data latih, tetapi juga mampu melakukan prediksi yang baik pada data yang belum dikenali.

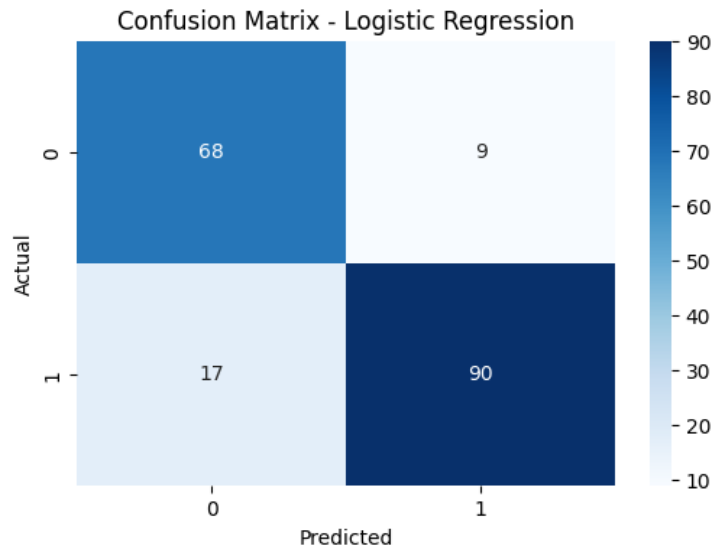
4) Evaluasi Model

a) Metrik Evaluasi

Dalam evaluasi model-model algoritma untuk dataset prediksi gagal jantung, digunakan 2 metrik utama, yaitu:

i. Confusion Matrix

❖ Logistic Regression



Gambar 4.1 *Confusion Matrix Logistic Regression*

Interpretasi:

- True Negatives (TN): 68 pasien tanpa penyakit jantung diprediksi dengan benar
- False Positives (FP): 9 pasien tanpa penyakit jantung diprediksi salah memiliki penyakit
- False Negatives (FN): 17 pasien dengan penyakit jantung tidak terdeteksi
- True Positives (TP): 90 pasien dengan penyakit jantung diprediksi dengan benar

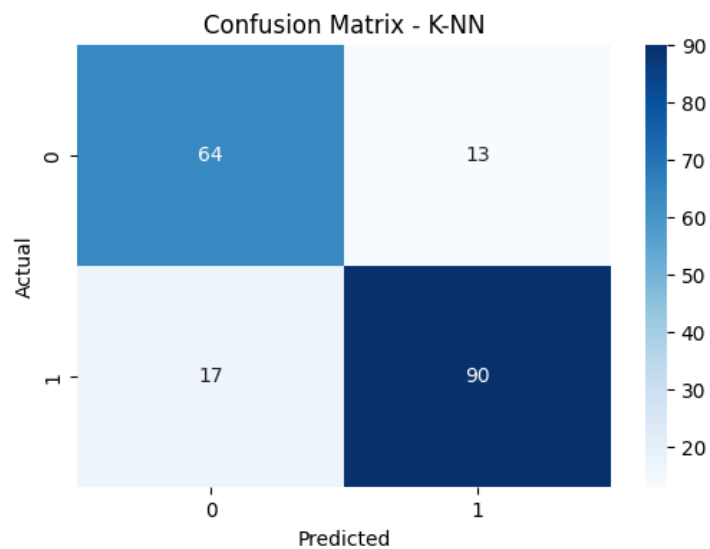
Berikut adalah tabel Classification Report dari model algoritma Logistic Regression.

Algoritma	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.858696	0.909091	0.841121	0.873786

Model cukup baik dalam mendeteksi pasien dengan penyakit jantung (84.11%) dan memiliki tingkat false alarm yang rendah (precision tinggi 90.9%). Namun terdapat 17 pasien dengan penyakit jantung tidak terdeteksi (false negative), yang dalam konteks medis bisa berbahaya karena pasien yang membutuhkan perawatan tidak mendapatkannya. Meskipun accuracy sudah baik (85.87%), model masih perlu ditingkatkan terutama untuk mengurangi false negatives. Dalam konteks medis, meningkatkan recall mungkin lebih penting daripada precision, karena lebih baik melakukan pemeriksaan

tambahan pada pasien sehat (false positive) daripada tidak mendeteksi pasien yang sakit (false negative).

❖ K-Nearest Neighbor (K-NN)



Gambar 4.2 *Confusion Matrix K-NN*

Interpretasi:

- True Negatives (TN): 64 pasien tanpa penyakit jantung diprediksi dengan benar
- False Positives (FP): 13 pasien tanpa penyakit jantung diprediksi salah memiliki penyakit
- False Negatives (FN): 17 pasien dengan penyakit jantung tidak terdeteksi
- True Positives (TP): 90 pasien dengan penyakit jantung diprediksi dengan benar

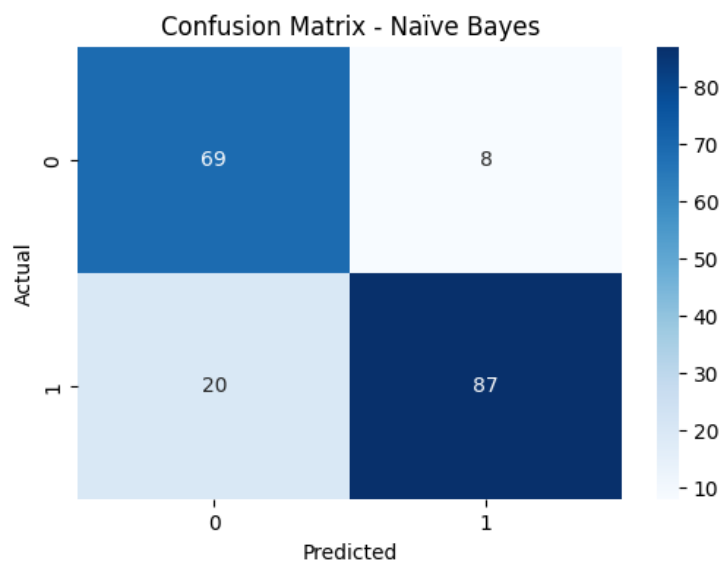
Berikut adalah tabel Classification Report dari model algoritma K-NN.

Algoritma	Accuracy	Precision	Recall	F1-Score
K-NN	0.836957	0.873786	0.841121	0.857143

Model K-NN menunjukkan performa yang cukup baik dengan recall 84.11%, sama dengan Logistic Regression, artinya kemampuan mendeteksi pasien dengan penyakit jantung setara. Namun, precision-nya lebih rendah (87.38%) karena menghasilkan lebih banyak false positive (13 dibandingkan 9

pada Logistic Regression). Accuracy keseluruhan sedikit lebih rendah (83.7%) dibandingkan Logistic Regression. Model ini juga memiliki 17 kasus false negative, yang berarti sejumlah pasien yang membutuhkan perawatan tidak teridentifikasi. Dalam konteks medis, model ini masih memerlukan perbaikan untuk mengurangi jumlah false positive dan false negative, meskipun secara keseluruhan kinerjanya cukup baik dengan F1-Score 85.71%.

❖ Naive Bayes



Gambar 4.3 *Confusion Matrix Naive Bayes*

Interpretasi:

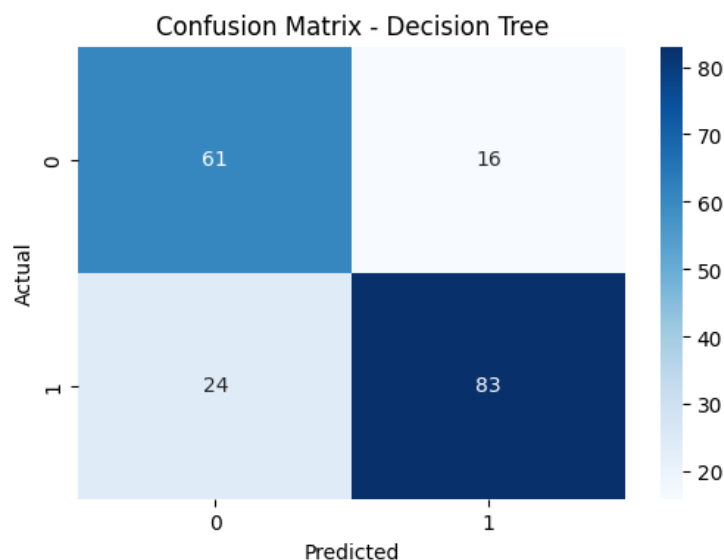
- True Negatives (TN): 69 pasien tanpa penyakit jantung diprediksi dengan benar
- False Positives (FP): 8 pasien tanpa penyakit jantung diprediksi salah memiliki penyakit
- False Negatives (FN): 20 pasien dengan penyakit jantung tidak terdeteksi
- True Positives (TP): 87 pasien dengan penyakit jantung diprediksi dengan benar

Berikut adalah tabel Classification Report dari model algoritma Naive Bayes.

Algoritma	Accuracy	Precision	Recall	F1-Score
Naive Bayes	0.847826	0.915789	0.813084	0.861386

Model Naïve Bayes memiliki precision tertinggi di antara semua model (91.58%), yang berarti ketika model ini memprediksi pasien positif penyakit jantung, prediksi tersebut sangat dapat diandalkan. Namun, recall-nya lebih rendah (81.31%) karena terdapat 20 kasus false negative, lebih banyak dibandingkan model lainnya. Hal ini menunjukkan model lebih konservatif dalam memprediksi kasus positif. Accuracy keseluruhan cukup baik (84.78%), namun dalam konteks medis, jumlah false negative yang lebih tinggi bisa menjadi kekhawatiran karena berarti lebih banyak pasien yang sebenarnya sakit tidak mendapatkan diagnosis yang tepat. Model ini menghasilkan F1-Score 86.14%, yang menunjukkan keseimbangan yang baik antara precision dan recall, meski masih perlu perbaikan dalam hal recall.

❖ Decision Tree



Gambar 4.4 *Confusion Matrix Decision Tree*

Interpretasi:

- True Negatives (TN): 61 pasien tanpa penyakit jantung diprediksi dengan benar
- False Positives (FP): 16 pasien tanpa penyakit jantung diprediksi salah memiliki penyakit
- False Negatives (FN): 24 pasien dengan penyakit jantung tidak terdeteksi
- True Positives (TP): 83 pasien dengan penyakit jantung diprediksi dengan benar

Berikut adalah tabel Classification Report dari model algoritma Decision Tree.

Algoritma	Accuracy	Precision	Recall	F1-Score
Decision tree	0.782609	0.838384	0.775701	0.805825

Model Decision Tree memiliki kinerja terendah di antara keempat model dengan accuracy 78.26%. Model ini menunjukkan precision 83.84% dan recall 77.57%, keduanya terendah di antara semua model. Terdapat 24 kasus false negative, yang menunjukkan model ini paling sering gagal mendeteksi pasien dengan penyakit jantung. Selain itu, model juga menghasilkan 16 kasus false positive, tertinggi di antara semua model, yang berarti banyak pasien sehat yang salah didiagnosa memiliki penyakit jantung. F1-Score 80.58% juga terendah di antara semua model. Dalam konteks medis, model ini memerlukan perbaikan signifikan untuk mengurangi jumlah false negative dan false positive.

ii. Loss Value

Loss value mengukur seberapa baik atau buruk kinerja model, yaitu seberapa besar kesalahan model dalam memprediksi output yang benar. Salah satu bentuk loss yang sering digunakan dalam klasifikasi adalah Log-Loss (juga dikenal sebagai Cross-Entropy Loss), yang mengukur perbedaan antara probabilitas yang diprediksi oleh model dan nilai label yang sebenarnya.

Log-Loss mengukur seberapa jauh prediksi probabilitas model dari label yang sebenarnya. Semakin kecil nilai log-loss, semakin baik model dalam memprediksi hasil yang benar.

Log Loss untuk Regresi Logistik: 0.3556, yang mana menunjukkan bahwa prediksi model cukup baik karena nilai log-loss mendekati nol. Artinya, prediksi probabilitas yang dihasilkan oleh model Logistic Regression relatif dekat dengan label yang sebenarnya.

b) Perbandingan Kinerja Model

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.858696	0.909091	0.841121	0.873786
K-NN	0.836957	0.873786	0.841121	0.857143
Naïve Bayes	0.847826	0.915789	0.813084	0.861386
Decision Tree	0.782609	0.838384	0.775701	0.805825

Perbandingan Semua Model:

- Accuracy: Logistic Regression (85.87%) > Naïve Bayes (84.78%) > K-NN (83.70%) > Decision Tree (78.26%)
- Precision: Naïve Bayes (91.58%) > Logistic Regression (90.91%) > K-NN (87.38%) > Decision Tree (83.84%)
- Recall: Logistic Regression & K-NN (84.11%) > Naïve Bayes (81.31%) > Decision Tree (77.57%)
- F1-Score: Logistic Regression (87.38%) > Naïve Bayes (86.14%) > K-NN (85.71%) > Decision Tree (80.58%)

Urutan model berdasarkan kinerja terbaik hingga terburuk:

1. Logistic Regression

Memiliki performa paling seimbang dengan nilai tertinggi pada Accuracy, Recall (bersama K-NN), dan F1-Score.

2. Naïve Bayes

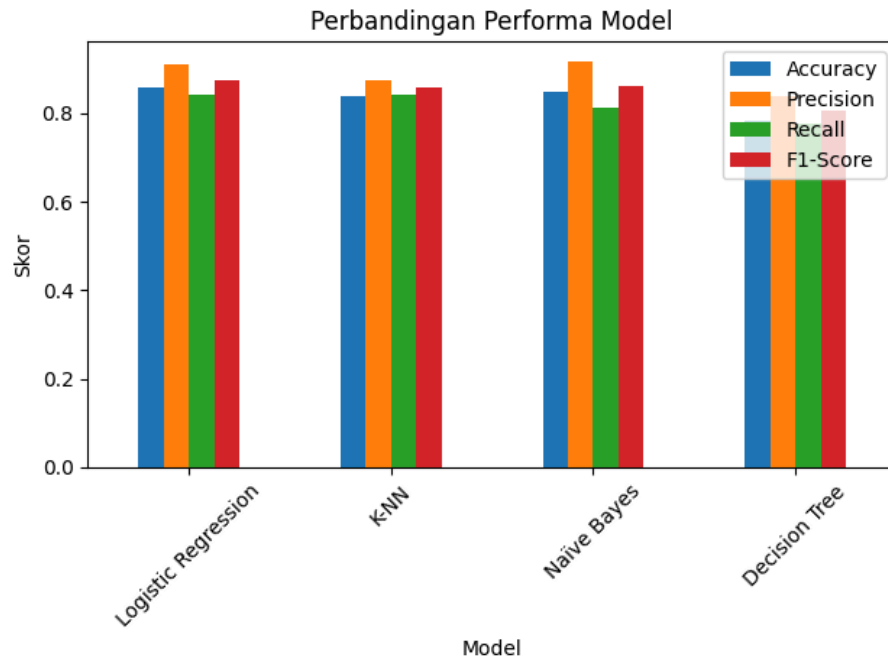
Unggul dalam Precision, namun sedikit lebih rendah pada Recall dibanding Logistic Regression dan K-NN.

3. K-Nearest Neighbors (K-NN)

Memiliki nilai Recall yang tinggi, tetapi Precision dan F1-Score berada di bawah dua model sebelumnya.

4. Decision Tree

Menunjukkan performa paling rendah di keempat metrik evaluasi (Accuracy, Precision, Recall, dan F1-Score).

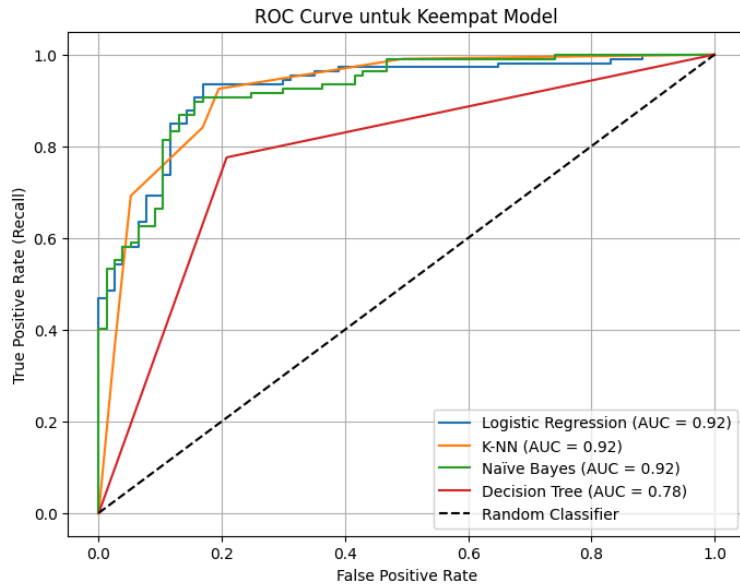


Gambar 4.5 *Perbandingan Performa Model Algoritma*

Secara keseluruhan, Logistic Regression menunjukkan kinerja terbaik dengan keseimbangan yang baik antara precision dan recall, serta accuracy tertinggi. Naïve Bayes memiliki precision tertinggi tetapi recall yang lebih rendah. Decision Tree memiliki kinerja terendah di semua metrik. Dalam konteks medis, model dengan recall tinggi lebih diutamakan untuk mengurangi false negative, sehingga Logistic Regression dan K-NN mungkin lebih sesuai untuk aplikasi diagnosis penyakit jantung.

5) Analisis Hasil

a) Grafik Fit



Gambar 5.1 ROC Curve

Semakin tinggi nilai AUC (Area Under the Curve), semakin baik kemampuan model dalam membedakan antara dua kelas, yaitu pasien dengan penyakit jantung (label 1) dan tanpa penyakit jantung (label 0). Berikut ini interpretasi untuk masing-masing model:

- Logistic Regression (AUC = 0.92)

Memiliki AUC tertinggi, menunjukkan performa terbaik dalam membedakan pasien dengan dan tanpa penyakit jantung.

- K-Nearest Neighbors (K-NN) (AUC = 0.92)

AUC sama dengan Logistic Regression, menunjukkan performa yang setara dalam hal kemampuan diskriminasi.

- Naïve Bayes (AUC = 0.92)

Juga memiliki AUC yang sama dengan dua model sebelumnya, menunjukkan performa yang sangat baik dan setara dalam klasifikasi.

- Decision Tree (AUC = 0.78)

Memiliki AUC terendah di antara keempat model. Meskipun begitu, nilainya masih jauh di atas 0.5, yang berarti performanya tetap lebih baik dari model acak (random guessing), namun kurang optimal dibanding model lainnya.

b) Kesimpulan

Model machine learning yang diimplementasikan telah menunjukkan kemampuan yang baik dalam memprediksi penyakit jantung. Regresi Logistik muncul sebagai model terbaik dengan akurasi 85,87%, presisi 90,91%, recall 84,11%, dan AUC 0,92. Kombinasi metrik ini menunjukkan bahwa model mampu mengidentifikasi sebagian besar pasien dengan penyakit jantung sambil meminimalkan false positives.

Performa model ini cukup menjanjikan untuk implementasi dalam sistem pendukung keputusan klinis, khususnya sebagai alat screening awal untuk mengidentifikasi pasien yang mungkin berisiko terkena penyakit jantung. Meskipun demikian, mengingat konsekuensi kesalahan diagnosis dalam konteks medis, model ini sebaiknya digunakan sebagai alat pendukung dan bukan pengganti penilaian klinis oleh dokter.